

Auditoria de seguridad y despliegue para VPS en Hostinger

Proyecto auditado: DesignSystemMissionControl | Fecha: 2026-03-10 | Solo auditoria, sin cambios de codigo

Resumen ejecutivo

Se reprodujo exactamente el mensaje observado en el VPS: 13 vulnerabilities (4 low, 6 moderate, 3 high). Ese conteo corresponde al arbol completo instalado y mezcla dependencias de produccion y desarrollo.

Al recalcular con `npm audit --omit=dev`, el riesgo efectivo de runtime baja a 8 vulnerabilidades: 2 high, 2 moderate y 4 low. Las dependencias mas sensibles en produccion son `express-rate-limit` y `multer`.

La razon mas probable de que Hostinger muestre las 13 es que el servidor esta instalando el arbol completo para compilar en el VPS o durante la etapa builder de Docker. El warning de `npm audit` no implica por si mismo que el deploy haya fallado, pero si confirma que hay trabajo pendiente de remediacion.

Metodologia

- `npm audit --json`: 13 vulnerabilidades totales, 4 low, 6 moderate, 3 high.
- `npm audit --omit=dev --json`: 8 vulnerabilidades de produccion, 4 low, 2 moderate, 2 high.
- `npm ls --depth=4`: `lodash` entra por `recharts`; `rollup` entra por `vite`.
- Revision de codigo y configuracion: `server/index.ts`, `server/middleware/upload.ts`, `server/controllers/digital-assets.ts`, `server/routes.ts`, `package.json`, `Dockerfile`, `DEPLOYMENT.md`, `ecosystem.config.cjs`, `db/index.ts`.

Por que aparece exactamente el conteo de 13 en el VPS

- `package.json` define un build que depende de `vite`, `esbuild` y `drizzle-kit`, por lo que un build local o en servidor necesita toolchain completo.
- `Dockerfile`: la etapa builder instala todas las dependencias con `npm ci` y la etapa final instala solo produccion con `npm ci --omit=dev`.
- `DEPLOYMENT.md` tiene una inconsistencia: recomienda `npm ci --omit=dev` y despues `npm run build`. Ese flujo no es compatible y empuja a hacer una instalacion completa en el VPS.

Hallazgos de produccion

- `express-rate-limit` 8.2.1 - High. Riesgo de bypass del rate limiting con IPv4 mapeado sobre IPv6. Evidencia: `server/index.ts:33-43`. Objetivo sugerido: 8.3.1.
- `multer` 2.0.2 - High. Riesgo de DoS por cleanup incompleto, agotamiento de recursos y recursion no controlada. Evidencia: `server/middleware/upload.ts:24-29`; `server/controllers/digital-assets.ts:29, 78, 119`. Objetivo sugerido: 2.1.1.
- `express` 4.21.2 con `body-parser` y `qs` transitivos - Low/Moderate. Riesgos de parsing y consumo de memoria. Evidencia: `server/index.ts:2 y 51-56`. Objetivo sugerido: 4.22.1.
- `express-session` 1.18.1 con `on-headers` - Low. Se encontro instalado en `package.json:73` pero sin uso activo en `server/`, `client/`, `shared/` ni `scripts/`. Recomendacion: remover si no es necesario; si se conserva, subir a 1.19.0.
- `lodash` 4.17.21 via `recharts` 2.15.4 - Moderate. Recomendacion: override a `lodash` 4.17.23 o

evaluar upgrade de recharts.

Hallazgos de toolchain / desarrollo

- vite 7.1.12 -> rollup 4.52.5 - High. Impacta build, no el runtime final. Objetivo sugerido: una combinacion con rollup 4.59.0; recomendado vite 7.3.1.
- drizzle-kit 0.31.4 -> @esbuild-kit/esm-loader -> esbuild - Moderate. Impacta migraciones y tooling. npm view muestra que 0.31.x sigue usando @esbuild-kit/esm-loader. Probar primero drizzle-kit 0.31.9 y, si el advisory persiste, aislarlo a CI o builder.

Revision de codigo y exposicion real

- Rate limiting: el limitador global esta en server/index.ts:33-43, pero PM2 corre en cluster segun ecosystem.config.cjs:5-6. El store en memoria no comparte conteos entre workers, por lo que el limite deja de ser estricto.
- No se encontro configuracion trust proxy. En Hostinger + Nginx esto puede degradar la precision de req.ip y afectar el rate limiting.
- Uploads: hay dos configuraciones separadas de multer. La de avatares limita tamano a 5 MB, pero no valida tipo ni cantidad. La de activos digitales no define limites explicitos de tamano o cantidad.
- Exposicion publica de archivos: server/routes.ts:37-42 sirve /uploads de forma publica con app.use("/uploads", express.static(uploadDir)).
- Dependencia no usada: express-session parece ser superficie instalada sin uso real.

Conexion y posibles errores a considerar en VPS

- Si se intenta compilar con solo dependencias de produccion, pueden aparecer errores como vite, esbuild o tsx no encontrados.
- server/index.ts ya valida JWT_SECRET, BASE_URL y PORT. Fallas posibles: Invalid runtime configuration e Invalid PORT.
- db/index.ts exige DATABASE_URL. Si la conexion falla, son esperables errores como ENOTFOUND, ECONNREFUSED, ETIMEDOUT o CERT_HAS_EXPIRED.
- Despues del hardening de uploads son esperables respuestas 413 Payload Too Large, 415 Unsupported Media Type y MulterError. Deben tratarse como efectos controlados, no como regresiones.
- Sin trust proxy y con cluster, el rate limit puede dar 429 inesperados o no aplicar correctamente.

Recomendaciones sin editar aun el codigo

- Actualizar primero express-rate-limit, multer y express; remover express-session si no se usa.
- Unificar el middleware de uploads con limites de tamano, cantidad, tipos permitidos, extensiones y limpieza ante fallo.
- Configurar trust proxy y usar un store compartido compatible con cluster para el rate limiting.
- Corregir el flujo de despliegue: build en CI y despliegue de artefacto, o Docker multi-stage, o build temporal con prune posterior; evitar npm ci --omit=dev seguido de build.

- Documentar el tratamiento de dependencias transitivas como lodash y drizzle-kit antes de la remediación.
- Ejecutar build, smoke tests, upload tests, login, health endpoint y npm audit --omit=dev --audit-level=high antes de promover a producción.

Plan de implementación propuesto

- Fase 0: congelar baseline, capturar lockfile actual y confirmar la vía de despliegue real usada en Hostinger.
- Fase 1: corregir el riesgo alto de runtime actualizando express-rate-limit y multer, y revisando compatibilidad de Express 4.22.1.
- Fase 2: revisar el código afectado con pruebas de uploads, acceso a /uploads y rate limit detrás de proxy.
- Fase 3: normalizar despliegue y corregir la guía oficial de deployment.
- Fase 4: prevención continua con pipeline de audit, SCA, smoke tests y health check con DB.

Manejo de errores para que no reaparezcan

- Centralizar un handler para MulterError con respuestas 400, 413 y 415 claramente diferenciadas.
- Registrar cada bloqueo de upload con usuario, ruta, tamaño y motivo.
- Agregar preflight de conectividad de base de datos al health check, no solo al proceso HTTP.
- Versionar el lockfile y ejecutar npm audit --omit=dev en CI en cada merge o release.
- Usar store compartido para rate limit si PM2 sigue en cluster.
- Hacer revisiones periódicas de dependencias no usadas para reducir superficie de ataque.
- Registrar un procedimiento único de build, rollback y despliegue para evitar instalaciones improvisadas en el VPS.

Evidencia clave revisada

- package.json: express 4.21.2, express-rate-limit 8.2.1, multer 2.0.2, express-session 1.18.1, recharts 2.15.4, drizzle-kit 0.31.4 y vite 7.1.9.
- server/index.ts: rate limiter global, validación de variables de entorno y handler global de errores.
- server/middleware/upload.ts: multer con límite de tamaño para avatar, sin filtro de tipos.
- server/controllers/digital-assets.ts: multer separado sin límites explícitos, rutas de carga múltiples.
- server/routes.ts: ruta pública /uploads y health endpoint básico.
- Dockerfile: builder con deps completas y runtime final con npm ci --omit=dev.
- DEPLOYMENT.md: inconsistencia entre instalación prod-only y build posterior.
- db/index.ts: fail-fast por DATABASE_URL, pero sin health check de conectividad real al arrancar.

Fuentes técnicas adicionales consultadas durante la auditoría: npm view para versiones objetivo de express, express-rate-limit, multer, express-session, vite, rollup, drizzle-kit, recharts y lodash.